

# FULL STACK JS

## Livret étudiant - Cahier des charges, travaux pratiques et rendu final

**Public :** étudiants de Master 1

**Durée :** 28 heures de formation, réparties sur quatre journées de 7 heures

**Format de travail prévu :** projet individuel, démonstrations courtes, exercices autonomes et soutenance

**Version :** document de travail du 24 septembre 2026, à valider avant diffusion définitive.

**À confirmer par l'établissement.** Le syllabus détaillé parle d'une **thématique imposée**. Le présent livret propose un choix encadré entre **TaskFlow, HabitLab et BudgetFlow** ; ce choix doit être approuvé par l'EFREI. Restent également à confirmer : mode de remise du dépôt (GitHub public ou privé, plateforme de dépôt, archive ZIP), déploiement cloud obligatoire ou non, barème et pondération de l'évaluation continue, calendrier des soutenances, et politique d'utilisation de l'IA. Les conventions API et les exercices de ce livret sont un **contrat pédagogique proposé par le formateur**, non une reproduction des règles officielles de l'établissement. Une fois approuvé et diffusé, ce contrat sera commun aux étudiants et au correcteur.

## 1. Votre mission

Vous allez concevoir, programmer, tester et documenter une application web complète en JavaScript. Elle comportera une interface React, une API Node.js/Express sécurisée par authentification et une base de données MongoDB. Vous réaliserez vous-même l'architecture, les écrans, les routes, la modélisation, la logique métier, les vérifications de qualité, les tests et la documentation.

**Objectif final :** un autre développeur doit pouvoir récupérer votre dépôt, comprendre l'application, l'installer, la lancer et reproduire les fonctionnalités annoncées. Lors de la soutenance individuelle, vous expliquerez également comment les trois couches communiquent et pourquoi les données d'un compte sont privées.

Le travail s'effectue sur un **seul** sujet choisi parmi les trois projets du catalogue ci-dessous, **sous réserve de validation du choix par l'établissement**. Il n'est pas demandé de développer les trois applications. Une démonstration réalisée en cours sert à illustrer une technique ; elle ne constitue pas un corrigé complet à recopier.

**MVP** signifie « produit minimum viable ». Dans ce module, le MVP désigne le **socle obligatoire et évalué** de votre projet. Les bonus ne sont ni nécessaires à la validation du MVP ni un moyen de remplacer une fonctionnalité obligatoire manquante.

### 1.1 Ce qui doit fonctionner à la fin

1. Un utilisateur peut s'inscrire et se connecter ; le serveur lui remet un JWT valide.
2. Le navigateur permet de créer, lister, consulter, modifier et supprimer les objets du sujet choisi.
3. Les opérations s'appuient sur MongoDB : les objets non supprimés restent disponibles après redémarrage de l'API.
4. Un compte ne peut pas accéder aux objets d'un autre, y compris via des requêtes HTTP directes.
5. L'API valide les entrées et renvoie les **routes, formats JSON et codes HTTP exacts** indiqués dans la partie « Contrat API ».

6. Le code est organisé, une suite de tests Jest/Supertest est exécutable et la documentation explique comment démarrer, utiliser et vérifier l'application.

## 2. Technologies et règles communes

| Couche           | Technologie / objectif                                                                                          |
|------------------|-----------------------------------------------------------------------------------------------------------------|
| Langage          | JavaScript.                                                                                                     |
| Interface        | React ; Vite est conseillé pour le projet. Écrans d'inscription, connexion et CRUD ; interface responsive.      |
| Back-end         | Node.js + Express ; API HTTP/JSON ; séparation des routes, contrôleurs, services et modèles.                    |
| Données          | MongoDB + Mongoose ; données durables et privées.                                                               |
| Authentification | bcrypt pour le hachage des mots de passe, JWT signé, middleware Bearer et vérification des droits côté serveur. |
| Qualité          | ESLint ; gestion des erreurs ; configuration par variables d'environnement.                                     |
| Tests            | Jest + Supertest ; cas nominaux, invalides et inter-utilisateurs.                                               |
| Documentation    | README.md, .env.example sans secret, Swagger/OpenAPI.                                                           |
| Outils           | npm, Git ; comprendre le rôle des outils de transpilation et de bundling, dont Babel/Webpack et Vite.           |

**Le projet est individuel.** Vous pouvez personnaliser le design, les composants et l'organisation interne du code, mais pas substituer une autre stack au socle du module : pas de Vue à la place de React, de NestJS à la place d'Express ni de PostgreSQL à la place de MongoDB. Les noms de routes, les noms de champs, les corps JSON et les codes HTTP ci-dessous sont contractuels.

**Hygiène du dépôt :** ne versionnez pas `node_modules`, `.env`, mots de passe, clés de signature JWT, jetons personnels, identifiants MongoDB réels ou données privées. Placez uniquement des *noms* de variables et valeurs fictives dans `.env.example`.

### 2.1 Comment travailler pendant les TP

À chaque étape : (1) lire le résultat attendu ; (2) réaliser l'implémentation ; (3) essayer un cas de réussite **et** un cas d'erreur ; (4) commiter un état stable ; (5) demander une aide ciblée si un blocage persiste. Les mécanismes présentés sur TaskFlow s'appliquent au sujet que vous avez retenu. Vous devez adapter la **ressource métier** sans modifier la **convention d'API** du sujet.

## 3. Catalogue fermé : trois sujets possibles

Vous reprenez **un seul** sujet. Une idée libre ou un quatrième projet ne fait pas partie du catalogue proposé. Chaque sujet comporte une ressource métier principale et les mêmes exigences transversales : compte utilisateur, API protégée, cinq opérations CRUD, React, MongoDB, validation, qualité, tests et documentation.

| Sujet                | Votre produit                                                  | Route métier obligatoire | MVP : objet principal                                                  | Extensions facultatives                                                            |
|----------------------|----------------------------------------------------------------|--------------------------|------------------------------------------------------------------------|------------------------------------------------------------------------------------|
| <b>A. TaskFlow</b>   | Suivre ses tâches personnelles.                                | /api/tasks               | Tâches privées avec titre, statut et options de description/ échéance. | Priorité et filtres ; suivi d'habitudes ; heatmap de contributions ; statistiques. |
| <b>B. HabitLab</b>   | Construire sa liste d'habitudes quotidiennes ou hebdomadaires. | /api/habits              | Habitudes privées avec titre, fréquence et statut d'activation.        | Journal d'accomplissement ; streak ; heatmap ; statistiques.                       |
| <b>C. BudgetFlow</b> | Suivre ses revenus et dépenses fictifs.                        | /api/transactions        | Transactions privées avec libellé, montant en centimes, type et date.  | Catégories ; solde agrégé ; budgets mensuels ; visualisations et export CSV.       |

### 3.1 Sujet A - TaskFlow

**Besoin :** gérer des tâches personnelles et suivre leur état d'avancement. Les utilisateurs ne voient que leurs tâches.

| Champ de la ressource Task | Format obligatoire                                                | À fournir lors d'un POST |
|----------------------------|-------------------------------------------------------------------|--------------------------|
| title                      | Chaîne non vide après trim, de 1 à 120 caractères.                | Oui                      |
| status                     | Exactement todo, doing ou done.                                   | Oui                      |
| description                | Chaîne facultative de 0 à 1000 caractères ; chaîne vide acceptée. | Non                      |
| dueDate                    | Date civile réelle YYYY-MM-DD ou null, facultative.               | Non                      |

**MVP obligatoire :** inscription/connexion, liste de ses tâches, ajout, détail, édition et suppression, validations et messages d'erreur, stockage persistant, autorisation côté API. Les filtres visuels ne sont pas nécessaires pour valider ce MVP.

**Bonus prédéfinis, facultatifs (ordre conseillé) :**

| Bonus | Extension           | Résultat attendu si vous la choisissez                                                                                                                 |
|-------|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------|
| B1    | Priorité et filtre  | Champ priority (low, medium, high), filtre par statut/échéance et compteur de tâches. Les routes additionnelles ne remplacent pas le CRUD obligatoire. |
| B2    | Habit tracker       | Une <b>seconde entité</b> Habit avec des événements datés de réalisation. Une tâche ponctuelle cochée n'est pas une habitude récurrente.               |
| B3    | Heatmap type GitHub | Agrégation quotidienne des tâches terminées et/ou réalisations d'habitudes ; grille calendrier, légende, jours à zéro et fuseaux horaires traités.     |

| Bonus | Extension    | Résultat attendu si vous la choisissez                                                                 |
|-------|--------------|--------------------------------------------------------------------------------------------------------|
| B4    | Statistiques | Taux de complétion hebdomadaire, évolution par période, calcul documenté et tests sur les agrégations. |

## 3.2 Sujet B - HabitLab

**Besoin :** gérer une liste personnelle d'habitudes quotidiennes ou hebdomadaires. Le MVP porte sur la **gestion des habitudes** ; enregistrer chaque réalisation dans un journal est **un bonus**, pas une obligation du MVP.

| Champ de la ressource Habit | Format obligatoire                                             | À fournir lors d'un POST |
|-----------------------------|----------------------------------------------------------------|--------------------------|
| title                       | Chaîne non vide après trim, de 1 à 120 caractères.             | Oui                      |
| frequency                   | Exactement daily ou weekly.                                    | Oui                      |
| active                      | Booléen true ou false. false est une valeur renseignée valide. | Oui                      |

**MVP obligatoire :** inscription/connexion, liste des habitudes du compte, ajout, détail, modification, suppression, activation/désactivation, validations, stockage persistant et protection des données.

**Bonus prédéfinis, facultatifs (ordre conseillé) :**

| Bonus | Extension                 | Résultat attendu si vous la choisissez                                                                                                  |
|-------|---------------------------|-----------------------------------------------------------------------------------------------------------------------------------------|
| B1    | Journal d'accomplissement | Entité HabitLog { habitId, date, completed }, réalisation/annulation et unicité par habitude et jour ou période hebdomadaire expliquée. |
| B2    | Streak                    | Série de périodes consécutives réussies ; règle pour les jours manqués et pour les habitudes hebdomadaires documentée.                  |
| B3    | Heatmap                   | Nombre d'habitudes accomplies par date, niveaux d'intensité, légende, cas sans données et fuseau horaire cohérent.                      |
| B4    | Statistiques              | Taux de réalisation sur 7/30 jours, graphiques et tests sur périodes sans données.                                                      |

## 3.3 Sujet C - BudgetFlow

**Besoin :** consigner des revenus et dépenses **fictifs**. Aucun accès bancaire et aucune transaction réelle ne sont demandés.

| Champ de la ressource Transaction | Format obligatoire                                 | À fournir lors d'un POST |
|-----------------------------------|----------------------------------------------------|--------------------------|
| label                             | Chaîne non vide après trim, de 1 à 120 caractères. | Oui                      |

| Champ de la ressource<br>Transaction | Format obligatoire                                                               | À fournir lors d'un POST |
|--------------------------------------|----------------------------------------------------------------------------------|--------------------------|
| amount                               | Entier JavaScript <b>sûr</b> , strictement positif, exprimé en <b>centimes</b> . | Oui                      |
| type                                 | Exactement income ou expense.                                                    | Oui                      |
| date                                 | Date civile réelle au format YYYY-MM-DD.                                         | Oui                      |

**MVP obligatoire** : inscription/connexion, liste de ses transactions, ajout, détail, modification, suppression, validation, stockage et isolation des données. Afficher les montants lisiblement dans l'interface : 12345 centimes doivent apparaître comme **123,45 €**. Dans l'API, amount reste l'entier 12345. Un montant est toujours positif : c'est type qui indique revenu ou dépense.

**Bonus prédéfinis, facultatifs (ordre conseillé) :**

| Bonus | Extension      | Résultat attendu si vous la choisissez                                                                            |
|-------|----------------|-------------------------------------------------------------------------------------------------------------------|
| B1    | Catégorisation | Catégorie à sélection contrôlée et filtres par type/date/catégorie.                                               |
| B2    | Solde agrégé   | Somme des revenus moins somme des dépenses du compte, en centimes exacts ; tests de précision et de période vide. |
| B3    | Budget mensuel | Limite par catégorie et par mois, avec alerte de dépassement et protection inter-utilisateurs.                    |
| B4    | Visualisation  | Graphiques des dépenses par catégorie et de leur évolution, export CSV traitant séparateur et dates.              |

**Règle générale sur les bonus** : commencez-les uniquement après avoir terminé le MVP. Ils ne dispensent jamais de réaliser une route, un écran, un test ou un livrable obligatoire. Si vous réalisez un bonus, définissez-en les critères d'acceptation et écrivez au moins un test dédié.

## 4. Contrat API v1 - règles exactes

Cette partie est la référence commune pour votre application et les futurs tests de correction. **Respectez les noms, méthodes, formats JSON et codes HTTP exactement tels qu'indiqués.** Les messages textuels d'erreur restent libres s'ils sont lisibles, non vides et sans informations sensibles.

### 4.1 Transport, identifiants, propriété

- Toutes les routes commencent par /api. Les requêtes possédant un corps et les réponses JSON utilisent Content-Type: application/json ; le contenu est en JSON UTF-8.
- Le serveur lit son port depuis la variable d'environnement PORT.
- L'endpoint de santé GET /api/health est public. Il retourne **200** avec le corps **exact** {"status": "ok"}.
- Toutes les routes métier requièrent Authorization: Bearer <JWT>. Sans JWT ou avec un JWT invalide, malformé ou expiré : **401**.
- Le propriétaire ownerId est calculé **exclusivement côté serveur à partir du JWT vérifié**. Le client ne choisit pas et ne modifie pas le propriétaire.

- La liste ne retourne **que** les ressources du compte connecté. Une tentative de lecture, modification ou suppression d'un objet appartenant à quelqu'un d'autre retourne **404**, comme pour un objet absent. Un identifiant malformé retourne **400**.
- L'identifiant public id est une **chaîne** contenant l'ObjectId MongoDB sérialisé. Le contrat public attend id, pas \_id. Les identifiants id et ownerId ne sont pas modifiables par PATCH.
- Les réponses métier peuvent contenir des propriétés supplémentaires **non sensibles**. Ne renvoyez jamais password, passwordHash, un JWT dans une ressource utilisateur ni de secrets.
- Les corps invalides, types incorrects, valeurs hors énumération, dates impossibles et tentatives de fournir ownerId ou id dans un POST/PATCH métier doivent être rejetés avec **400**.
- Un PATCH est **partiel et non vide**. Il n'accepte que les champs métier documentés. Une propriété inconnue, un objet vide ou un champ métier invalide ne doit pas être accepté.

## 4.2 Forme d'erreur commune

Toutes les erreurs applicatives sont renvoyées en JSON :

```
{"error":{"code":"INVALID_INPUT","message":"Message lisible"}}
```

| Situation                                              | HTTP | error.code obligatoire |
|--------------------------------------------------------|------|------------------------|
| Corps incorrect, champ ou identifiant malformé         | 400  | INVALID_INPUT          |
| Jeton absent/invalide/expiré ou connexion refusée      | 401  | UNAUTHORIZED           |
| Objet inexistant ou appartenant à un autre utilisateur | 404  | NOT_FOUND              |
| Email déjà utilisé                                     | 409  | EMAIL_ALREADY_USED     |

Les messages peuvent varier, mais doivent être non vides, compréhensibles et ne contenir ni trace serveur ni secret. Un utilisateur non authentifié ne reçoit pas les données privées.

## 4.3 Authentification - routes publiques

### POST /api/auth/register

#### Exemple de requête :

```
POST /api/auth/register HTTP/1.1
Content-Type: application/json

{"email":"alice@example.test","password":"MotDePasse123!"}
```

L'email doit être valide. Sa comparaison et son stockage sont **insensibles à la casse**. Le mot de passe comporte **au moins 8 caractères** et est stocké exclusivement sous la forme d'un hash bcrypt. Un email déjà utilisé retourne **409** et EMAIL\_ALREADY\_USED ; un corps invalide retourne **400** et INVALID\_INPUT.

**Succès : HTTP 201.** Schéma contractuel de la réponse (les valeurs de id et token ci-dessous sont des *substituts d'exemple*, pas des valeurs littérales à renvoyer) :

```
{"user":{"id":"<id>","email":"alice@example.test"},"token":"<jwt>"}
```

token doit contenir un vrai JWT signé et valide, user.id une chaîne. Aucun mot de passe ni hash dans la réponse.

## POST /api/auth/login

### Exemple de requête :

```
POST /api/auth/login HTTP/1.1
Content-Type: application/json

{"email": "alice@example.test", "password": "MotDePasse123!"}
```

**Succès : HTTP 200**, même schéma de réponse que pour register :

```
{"user": {"id": "<id>", "email": "alice@example.test"}, "token": "<jwt>"}
```

Mauvais identifiants : **401** et UNAUTHORIZED, sans révéler si l'email ou le mot de passe est incorrect. Corps invalide : **400**.

**Gestion du JWT** : la clé de signature vient d'une variable d'environnement serveur, par exemple JWT\_SECRET, jamais du front-end ou d'une valeur secrète committée. Vous devez pouvoir expliquer l'expiration du jeton et les compromis de son stockage dans le navigateur. Envoyer simplement un email depuis React ne constitue **pas** une authentification.

## 4.4 Routes métier - noms et méthodes obligatoires

La table suivante emploie :id comme **paramètre dynamique** de l'URL. Il est remplacé, lors d'une vraie requête, par l'id retourné par l'API.

| Projet     | Route collection exacte | Route objet exacte    |
|------------|-------------------------|-----------------------|
| TaskFlow   | /api/tasks              | /api/tasks/:id        |
| HabitLab   | /api/habits             | /api/habits/:id       |
| BudgetFlow | /api/transactions       | /api/transactions/:id |

| Méthode | Chemin              | Entrée                               | Succès contractuel                                                 |
|---------|---------------------|--------------------------------------|--------------------------------------------------------------------|
| GET     | /api/<resource>     | JWT                                  | <b>200</b> {"items": [<resource>, ...]} ; si vide : {"items": []}. |
| POST    | /api/<resource>     | JWT + champs requis du sujet         | <b>201</b> , objet créé.                                           |
| GET     | /api/<resource>/:id | JWT                                  | <b>200</b> , objet correspondant.                                  |
| PATCH   | /api/<resource>/:id | JWT + champs métier partiels valides | <b>200</b> , objet modifié.                                        |
| DELETE  | /api/<resource>/:id | JWT                                  | <b>204</b> , aucun corps.                                          |

**Substitution obligatoire** : <resource> vaut **exactement** tasks, habits ou transactions. Les alias /api/todos, /api/habit et /api/expenses **ne remplacent pas** ces routes, même s'ils sont fonctionnels. Des routes supplémentaires sont autorisées pour un bonus à condition de conserver intactes les cinq routes de base.

**Erreurs pour ces routes** : **400** pour un corps invalide ou un id malformé ; **401** si l'utilisateur n'est pas authentifié ; **404** si l'objet est absent ou appartient à un autre compte. Après un DELETE réussi, un GET sur cet id retourne **404**. Après redémarrage de l'API, une ressource non supprimée reste accessible à son propriétaire grâce à MongoDB.

### Exemple complet : créer, lister, modifier et supprimer une tâche

Exemple de **requête de création TaskFlow** avec le JWT A :

```
POST /api/tasks HTTP/1.1
Authorization: Bearer <JWT_A>
Content-Type: application/json
```

```
{"title":"Préparer la démo","status":"todo","description":"Plan et données","dueDate":"2026-10-05"}
```

**Succès : HTTP 201**, corps d'exemple conforme au schéma de l'objet créé :

```
{"id":"507f1f77bcf86cd799439011","title":"Préparer la démo","status":"todo","description":"Plan et données","dueDate":"2026-10-05"}
```

Exemple de **liste** : GET /api/tasks avec le même JWT, **HTTP 200** :

```
{"items":[{"id":"507f1f77bcf86cd799439011","title":"Préparer la démo","status":"todo","description":"Plan et données","dueDate":"2026-10-05"}]}
```

Exemple de **modification partielle** :

```
PATCH /api/tasks/507f1f77bcf86cd799439011 HTTP/1.1
Authorization: Bearer <JWT_A>
Content-Type: application/json
```

```
{"status":"done"}
```

**Succès : HTTP 200**, corps d'exemple :

```
{"id":"507f1f77bcf86cd799439011","title":"Préparer la démo","status":"done","description":"Plan et données","dueDate":"2026-10-05"}
```

DELETE /api/tasks/507f1f77bcf86cd799439011 avec le JWT A retourne **204 sans corps**. Une consultation ultérieure de cette même tâche retourne **404** avec NOT\_FOUND. Si le compte B tente de consulter, modifier ou supprimer une tâche encore existante du compte A, la réponse est également **404** avec NOT\_FOUND.

## Exemples de création HabitLab et BudgetFlow

HabitLab, **POST** /api/habits, corps de requête :

```
{"title":"Marcher","frequency":"daily","active":true}
```

**HTTP 201**, exemple d'objet créé :

```
{"id":"507f191e810c19729de860ea","title":"Marcher","frequency":"daily","active":true}
```

BudgetFlow, **POST** /api/transactions, corps de requête :

```
{"label":"Courses","amount":12345,"type":"expense","date":"2026-10-05"}
```

**HTTP 201**, exemple d'objet créé :

```
{"id":"507f1f77bcf86cd799439012","label":"Courses","amount":12345,"type":"expense","date":"2026-10-05"}
```

Dans les deux sujets, les autres méthodes GET/PATCH/DELETE reprennent **exactement** le contrat commun ci-dessus, avec la route correspondante. La liste utilise toujours l'enveloppe {"items":[...]}.

## 4.5 Règles de validation à implémenter côté serveur

| Ressource         | Champs et règles                                                                                                                                                                                  |
|-------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>TaskFlow</b>   | title trimé, 1 à 120 caractères ; status parmi todo, doing, done ; description facultative, 0 à 1000 caractères, chaîne vide admise ; dueDate facultative, date civile réelle YYYY-MM-DD ou null. |
| <b>HabitLab</b>   | title trimé, 1 à 120 caractères ; frequency parmi daily, weekly ; active booléen requis, y compris lorsque la valeur est false.                                                                   |
| <b>BudgetFlow</b> | label trimé, 1 à 120 caractères ; amount entier JavaScript sûr, strictement positif, en <b>centimes</b> ; type parmi income, expense ; date date civile réelle YYYY-MM-DD.                        |

### Exemples d'entrées invalides à refuser avec 400 / INVALID\_INPUT :

- TaskFlow : {"title": "Préparer", "status": "archived"}.
- HabitLab : {"title": "Marcher", "frequency": "hourly", "active": true}.
- BudgetFlow : {"label": "Courses", "amount": 12.34, "type": "expense", "date": "2026-10-05"} ; 0, un montant négatif ou une chaîne de caractères sont également invalides pour amount.
- Toutes ressources : un titre ou libellé vide après trim, une date impossible, ownerId/id envoyé dans la création ou la modification, ou un PATCH vide.

**Important** : valider les formulaires dans React aide l'utilisateur, mais ne protège pas l'API. La validation serveur reste obligatoire, y compris si un client HTTP contourne l'interface.

## 5. Planning du module : quatre jours, 28 heures

Les durées ci-dessous servent de **guide pédagogique**, non de calendrier institutionnel définitif. Vous progressez sur **votre** sujet en parallèle des démonstrations. Les points de contrôle servent à signaler les blocages ; leur éventuelle pondération dans l'évaluation continue sera précisée après validation de l'établissement.

| Jour            | Fil conducteur                                                                                                       | Travail à terminer / point de contrôle                                       |
|-----------------|----------------------------------------------------------------------------------------------------------------------|------------------------------------------------------------------------------|
| <b>J1 - 7 h</b> | Architecture front/API/base, Express, HTTP/REST, routes/contrôleurs/services, MongoDB/Mongoose, validations et CRUD. | API métier fonctionnelle et persistante, vérifiée au moyen d'un client HTTP. |
| <b>J2 - 7 h</b> | Compte utilisateur, bcrypt, JWT, middleware Bearer, autorisation A/B, React, formulaires et consommation de l'API.   | Deux comptes cloisonnés et un CRUD complet utilisable dans le navigateur.    |
| <b>J3 - 7 h</b> | Clean Code, refactoring, ESLint, tests Jest/Supertest, Swagger/OpenAPI, README, build et préparation au déploiement. | Tests reproductibles, code analysé, documentation exploitable.               |
| <b>J4 - 7 h</b> | Recette, corrections, rendu traçable et maintenance individuelle.                                                    | Version finale identifiable, démonstration du MVP et explication du code.    |

**Évaluation orale** : le syllabus détaillé prévoit une **présentation individuelle de 10 minutes, suivie de questions**. L'horaire exact et l'organisation des passages seront communiqués après confirmation des effectifs.

## 6. Travaux pratiques progressifs

### Jour 1 - Architecture, Express et persistance

#### TP 1 - Initialisation, santé et architecture

**À construire** : créez votre dépôt avec frontend/ et backend/ ; initialisez React/Vite et Express ; séparez la création de l'application Express du démarrage du serveur ; activez le parsing JSON ; exposez GET /api/health ; esquissez le chemin route -> contrôleur -> service -> modèle ; configurez .gitignore et .env.example sans secrets. Votre page React initiale peut afficher un simple titre.

**À vérifier** : les deux applications démarrent indépendamment ; la route de santé renvoie **200** et {"status":"ok"} ; importer le module Express pour les futurs tests ne lance pas le serveur ; aucun secret n'est versionné.

#### TP 2 - Première route métier

**À construire** : choisissez tasks, habits ou transactions. Créez les routes, contrôleurs et services. Implémentez provisoirement GET collection et POST avec une liste en mémoire pour comprendre le trajet d'une requête. Lisez le JSON d'entrée, appliquez les premières validations et renvoyez les formats HTTP/JSON du contrat.

**À vérifier** : route exacte, création et liste fonctionnelles, corps invalide rejeté avec l'erreur JSON contractuelle. **La liste en mémoire est temporaire : elle doit disparaître du produit final.**

#### TP 3 - MongoDB et CRUD persistant

**À construire** : définissez le schéma Mongoose de votre ressource ; remplacez le stockage temporaire par MongoDB ; implémentez GET par id, PATCH et DELETE ; centralisez la gestion des erreurs. Prévoyez une propriété serveur ownerId qui sera reliée au compte lors du J2.

**Prudence pendant J1** : tant que l'authentification n'est pas implémentée, utilisez uniquement une base et des données de développement **locales** ; ne déployez pas publiquement une API exposant des données privées.

**À vérifier** : CRUD sur données réelles ; **201/200/204, 400** pour un champ invalide, **404** pour un id valide absent ; donnée non supprimée conservée après redémarrage ; aucune propriété ownerId du client n'est acceptée comme autorité.

**Défi autonome** : TaskFlow rejette status:"archived" ; HabitLab rejette frequency:"hourly" ; BudgetFlow rejette amount:12.34.

**Checkpoint J1** : démontrez POST, GET, PATCH, DELETE et un cas 400 ; identifiez le fichier responsable de la validation.

### Jour 2 - Authentification, autorisation et React

#### TP 4 - Comptes utilisateurs et JWT

**À construire** : modèle User avec email unique et hash bcrypt ; validation email/mot de passe (8 caractères minimum) ; POST /api/auth/register et POST /api/auth/login ; signature du JWT avec une variable d'environnement ; middleware vérifiant l'en-tête Bearer ; protection de toutes les routes métier.

**À vérifier** : register **201**, login **200**, mauvais mot de passe **401**, email déjà utilisé **409**, ressource métier appelée sans jeton **401** ; aucun mot de passe/hash dans les réponses.

## TP 5 - Isolation de deux comptes

**À construire :** lors du POST métier, associez l'objet à l'utilisateur issu du JWT vérifié ; filtrez GET collection sur cet utilisateur ; conditionnez GET détail, PATCH et DELETE au propriétaire. Créez les comptes A et B et des objets distincts pour chacun.

**À vérifier :** le compte B ne voit aucun objet A dans sa liste ; les requêtes GET/PATCH/DELETE de B sur l'id d'un objet A encore existant renvoient **404** ; tenter de choisir ownerId via JSON échoue ; un JWT falsifié renvoie **401**.

## TP 6 - Interface React du MVP

**À construire :** vues inscription, connexion, liste, création, détail/édition ; navigation et état de session ; appels fetch avec Bearer ; gestion du chargement, des succès/erreurs ; suppression avec confirmation. Utilisez des labels explicites, des boutons nommés, des erreurs lisibles et un focus visible ; limitez les débordements sur mobile.

**À vérifier :** parcours complet depuis le navigateur, nouveau compte -> connexion -> création -> consultation -> modification -> suppression. Après rechargement, les données sont toujours côté serveur ; la stratégie de session est expliquée. **Masquer un bouton côté React ne remplace jamais une autorisation côté API.**

**Checkpoint J2 :** montrez les comptes A et B, puis expliquez pourquoi la sécurité est appliquée dans Express et pas seulement dans React.

## Jour 3 - Qualité, tests et documentation

### TP 7 - Refactoring, ESLint et build

**À construire :** clarifiez les noms ; isolez les règles métier dans les services ; retirez les doublons ; ne laissez aucun secret en dur ; traitez les erreurs asynchrones ; activez et exécutez ESLint ; vérifiez le build React.

**À vérifier :** les scénarios fonctionnent après refactoring ; lint et build sont reproductibles à partir des commandes du README ; aucune donnée sensible dans le dépôt.

### TP 8 - Tests Jest et Supertest

**À construire :** exposez une application Express importable sans ouverture de port ; utilisez une **base de test isolée** de la base de développement ; écrivez des tests couvrant au minimum : (1) un parcours CRUD nominal ; (2) une donnée métier invalide -> **400** ; (3) un accès sans JWT -> **401** ; (4) B ne peut pas lire ni modifier un objet appartenant à A. Créez et nettoyez vos fixtures de manière déterministe.

**À vérifier :** la commande de test documentée exécute la suite ; aucune donnée personnelle ni base de développement n'est utilisée ; la suppression de la protection inter-utilisateurs fait échouer le test correspondant.

### TP 9 - Swagger, README et exploitation

**À construire :** décrivez OpenAPI (Bearer, auth, toutes les routes CRUD, schémas de requête/réponse, erreurs applicables **400/401/404/409**) ; rédigez le README : besoin, stack, versions, installation, environnement, lancement MongoDB, lancement front/back, tests/lint/build, architecture, choix et limites. Expliquez le rôle de Vite, Babel et Webpack ainsi que l'emplacement des tests dans une future chaîne CI/CD. Préparez un essai de déploiement **selon les moyens validés par le formateur**.

**À vérifier :** un camarade peut suivre le README sans inventer de commande ; Swagger décrit **vos vraies routes** et non des exemples fictifs ; les commandes de qualité tournent réellement.

**Checkpoint J3 :** présentez les résultats des tests, du lint et du build, puis faites retrouver et utiliser un endpoint dans OpenAPI.

## Jour 4 - Recette, rendu et soutenance

### TP 10 - Recette finale

Exécutez la checklist de la partie 7, avec **deux comptes**, plusieurs entrées invalides et un redémarrage de l'API. Corrigez dans cet ordre : défauts de sécurité, routes non conformes, CRUD et démarrage ; puis qualité et documentation. **N'entreprenez un bonus que si tout le socle obligatoire est terminé.**

### TP 11 - Remise traçable

Créez un commit final et relevez son **SHA**. Préparez le dépôt Git individuel et, **uniquement si cette modalité est confirmée**, le lien GitHub demandé et une archive ZIP de la même version. Ne joignez ni `node_modules`, ni `.env`, ni `secret`, ni données réelles. Indiquez votre sujet et comment lancer le front-end et le back-end. Une fois la version gelée, ne la modifiez pas sans instruction explicite du formateur.

### TP 12 - Soutenance individuelle

Préparez **10 minutes** : objectif et sujet, architecture et circulation des données, démonstration du MVP, choix de validation/sécurité, un test automatisé et la documentation, limites et éventuels bonus. Répondez aux questions portant sur **votre propre code**. L'ordre de passage et la durée des questions restent à préciser.

## 7. Checklist de recette - à exécuter avant le rendu

Cochez chaque étape seulement après l'avoir **réellement essayée**.

### Installation et démarrage

- ☐ Le dépôt contient le code source React et Express et ne contient ni secrets ni `node_modules`.
- ☐ Une personne peut suivre le `README.md` et configurer les variables via `.env.example`.
- ☐ MongoDB, le serveur et le front démarrent en suivant les commandes documentées.
- ☐ `GET /api/health` répond **200** avec exactement `{"status": "ok"}`.
- ☐ Le build React, le lint et la commande de tests documentée s'exécutent.

### Authentification et droits

- ☐ Inscription A : **201**, objet `user` et JWT signé, sans mot de passe ou hash exposé.
- ☐ Connexion A : **200** et JWT ; mauvaise combinaison email/mot de passe : **401**.
- ☐ Email déjà utilisé : **409** ; mot de passe trop court ou email invalide : **400**.
- ☐ Sans JWT ou avec JWT falsifié, toute route métier refuse l'accès avec **401**.
- ☐ Créer aussi le compte B ; sa liste ne contient **aucun** objet A.
- ☐ Sur l'id d'un objet A encore existant, B reçoit **404** en GET, PATCH et DELETE.
- ☐ Envoyer `id` ou `ownerId` dans un POST/PATCH métier est refusé avec **400**.

### CRUD, données et erreurs

- ☐ Les cinq routes exactes de votre sujet sont implémentées avec leurs méthodes HTTP exactes.
- ☐ GET collection répond **200** avec `{"items": []}` si la collection du compte est vide.
- ☐ POST d'un objet valide répond **201** et retourne un objet dont `id` est une chaîne.
- ☐ GET par id et PATCH valide répondent **200** et retournent les objets attendus.
- ☐ PATCH partiel valide fonctionne ; PATCH vide ou champ interdit/invalide donne **400**.

- ☐ DELETE répond **204 sans corps** ; GET sur l'id supprimé donne **404**.
- ☐ Un id malformé donne **400** ; un id valide absent donne **404**.
- ☐ Les champs, valeurs et règles de validation de **votre** sujet sont respectés.
- ☐ Les réponses applicatives d'erreur utilisent `error.code` et `error.message` selon le contrat.
- ☐ Une donnée non supprimée reste disponible après redémarrage du serveur.

## Front-end, qualité et documentation

- ☐ Inscription, connexion et CRUD sont réalisables **dans React**, pas seulement dans un client HTTP.
- ☐ Les formulaires affichent des erreurs lisibles ; chargement, succès et suppression sont compréhensibles.
- ☐ L'interface reste utilisable sur un écran mobile ; les contrôles principaux sont accessibles au clavier.
- ☐ Les tests Jest/Supertest couvrent le nominal, l'entrée invalide, l'absence de JWT et l'isolation A/B.
- ☐ README.md contient toutes les commandes réellement nécessaires, les choix d'architecture et les limites.
- ☐ Swagger/OpenAPI reflète les routes, schémas et codes d'erreur **réellement implémentés**.
- ☐ Le commit SHA final est identifiable et la version démontrée correspond à celle rendue.

# 8. Livrables et documentation

## 8.1 Ce que vous préparez

1. **Un dépôt Git individuel** contenant tout le code du front-end, du back-end et les tests ; historique Git et commit SHA final identifiables.
2. **Un README.md détaillé** : objectif, sujet, versions et prérequis, installation, `.env.example`, MongoDB, lancement du front et de l'API, URLs/ports, tests/lint/build, architecture, choix techniques et limites connues.
3. **Une documentation Swagger/OpenAPI** (fichier ou interface clairement accessible) décrivant l'authentification Bearer, les deux routes auth, la route de santé, les cinq opérations métier, les schémas des données et les erreurs correspondantes.
4. **Une application démontrable** depuis le navigateur et une suite de tests Jest/Supertest effectivement exécutable.
5. **Une soutenance individuelle** de 10 minutes de présentation suivies de questions.

La convention `frontend/` et `backend/` est **envisagée** pour les dépôts. Les noms de scripts `npm install`, `npm run dev`, `npm run build`, `npm run lint`, `npm test` sont des **suggestions**, pas des commandes garanties identiques pour tous tant que le mode de remise et les conventions d'environnement ne sont pas finalisés. **Documentez précisément les commandes réelles de votre projet**. Aucun dépôt de démarrage n'est annoncé ou requis par le présent livret : vous construisez vous-même le projet.

## 8.2 Démonstration orale : déroulé indicatif des 10 minutes

| Durée proposée | Contenu                                                                      |
|----------------|------------------------------------------------------------------------------|
| 1 min          | Sujet choisi, problème résolu et MVP réalisé.                                |
| 2 min          | Architecture React -> Express -> MongoDB ; dossiers et chemin d'une requête. |
| 3 min          | Démonstration réelle : compte, CRUD et état des données.                     |
| 2 min          | JWT, hash du mot de passe, validation et isolation des comptes A/B.          |

| Durée proposée | Contenu                                                                        |
|----------------|--------------------------------------------------------------------------------|
| 1 min          | Exemple de test, Swagger et README.                                            |
| 1 min          | Limites identifiées, améliorations futures et, le cas échéant, bonus réalisés. |

Préparez-vous à expliquer pourquoi le serveur filtre par propriétaire, la différence entre **401** et **404**, ce qui se passe en cas de redémarrage, ou comment vos tests détectent une régression. Les réponses doivent correspondre à **vosre implémentation**.

## 9. Évaluation - proposition de barème, non officielle

Le syllabus détaillé prévoit une **évaluation continue et une soutenance finale individuelle**, mais **ne fournit pas de barème chiffré**. Le tableau ci-dessous est une **proposition du formateur**, qui ne deviendra applicable qu'après validation et communication aux étudiants.

| Critère proposé                                    | Points proposés | Ce qui sera observé                                                                                                                                                                                                    |
|----------------------------------------------------|-----------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Front-end et intégration                           | <b>4</b>        | React, écrans, appels à l'API, CRUD dans l'interface, erreurs et responsive.                                                                                                                                           |
| API, persistance et contrat                        | <b>4</b>        | Routes et méthodes exactes, formats/codes HTTP, Mongoose, CRUD et persistance. <b>Jusqu'à 1,5 point au sein de ces 4</b> pourrait porter sur routes/méthodes et réponses, répartition précise à fixer avant diffusion. |
| Authentification, autorisation et validation       | <b>4</b>        | bcrypt, JWT vérifié, isolation A/B, validations serveur et erreurs contrôlées.                                                                                                                                         |
| Architecture, qualité et tests                     | <b>3</b>        | Modularité, séparation des responsabilités, ESLint, Jest/Supertest et configuration reproductible.                                                                                                                     |
| README et Swagger/OpenAPI                          | <b>2</b>        | Documentation exploitable et conforme au projet.                                                                                                                                                                       |
| Préparation à l'exploitation / build / déploiement | <b>1</b>        | Build et configuration ; composante déploiement <b>selon la décision finale de l'EFREI</b> .                                                                                                                           |
| Soutenance individuelle                            | <b>2</b>        | Démonstration, compréhension du code et justification des choix.                                                                                                                                                       |
| <b>Total indicatif</b>                             | <b>20</b>       | <b>Sous réserve de validation par l'établissement.</b>                                                                                                                                                                 |

**Routes obligatoires** : un endpoint nommé autrement (/api/todos au lieu de /api/tasks, par exemple) ne respecte pas le contrat et échouera aux tests visant **cette** route. La perte de points éventuelle relève du critère API selon le **barème validé et annoncé** ; le même renommage ne doit pas entraîner une **double pénalité**. Un incident du correcteur ou de l'environnement n'est pas automatiquement un échec du projet : il nécessite une relance ou une vérification humaine.

**Bonus :** ils ne donnent **pas automatiquement** des points supplémentaires et ne compensent pas un MVP incomplet. Leur prise en compte éventuelle dans la note doit être fixée et annoncée à l'avance. La pondération de l'évaluation continue et la règle de notation finale restent à confirmer.

## 10. À confirmer par l'établissement

**Ces points ne sont pas encore des consignes officielles.** Le formateur transmettra une version finale et datée dès qu'ils auront été arbitrés. Ne déduisez pas une obligation du seul fait qu'une option est décrite dans ce livret.

1. **Sujet** : confirmation du choix entre les trois thématiques préétablies alors que le syllabus parle de « thématique imposée » ; le catalogue pourrait être ajusté selon la décision de l'EFREI.
2. **Remise** : GitHub public/privé, archive ZIP complémentaire, plateforme de dépôt, date et heure de gel du commit. Un dépôt **Git** et sa documentation sont bien prévus dans la fiche d'épreuve, mais le caractère public ou la présence d'un ZIP ne sont pas établis.
3. **Déploiement** : le plan de cours mentionne le cloud ; reste à décider si une URL déployée constitue un **livrable individuel obligatoire**, et avec quels accès/outils.
4. **Évaluation** : barème sur 20, répartition exacte des points et pondération de l'évaluation continue ; prise en compte éventuelle des bonus ; modalités et durée des questions après les 10 minutes de présentation.
5. **Usage de l'IA** : outils autorisés/interdits, déclaration de leur utilisation et règles applicables à l'épreuve.
6. **Organisation pratique** : taille des groupes, horaires précis, matériel, accès MongoDB/cloud, ordre et créneaux des soutenances.

### Mini-glossaire

**API REST** : interface HTTP permettant d'interagir avec les ressources. **CRUD** : créer, lire, modifier et supprimer. **JWT** : jeton signé utilisé ici pour identifier un utilisateur après authentification.

**Authentification** : établir qui effectue la requête. **Autorisation** : déterminer si cet utilisateur a le droit d'accéder à cet objet. **Mongoose** : bibliothèque de modélisation et d'accès à MongoDB depuis Node.js.

**OpenAPI/Swagger** : description structurée de l'API. **MVP** : ensemble minimal **obligatoire** des fonctions demandées dans le projet.

*Sources de conception du livret : énoncé commun étudiant, catalogue des trois projets, contrat API v1, TP progressifs sur 4 jours et document de rendu/évaluation/soutenance. Le syllabus EFREI détaillé reste la référence des objectifs et modalités institutionnelles ; les conventions techniques et le barème proposé sont identifiés comme tels.*